

[Arquitectura de Computadoras]

SISTEMAS NUMÉRICOS

Ing. Gustavo Maurokefalidis

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- El valor del número se calcula mediante el polinomio:

$$N \equiv ... + n_3 \cdot b_3 + n_2 \cdot b_2 + n_1 \cdot b_1 + n_0 \cdot b_0 + n_{-1} \cdot b_{-1} ...$$

$$N \equiv \sum_2 2_2 \cdot 10^2$$

- Ejemplo:

$$3278,52 = 3 \cdot 10^3 + 2 \cdot 10^2$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- El valor del número se calcula mediante el polinomio:

$$N \equiv ... + n_3 \cdot b_3 + n_2 \cdot b_2 + n_1 \cdot b_1 + n_0 \cdot b_0 + n_{-1} \cdot b_{-1} ...$$

$$N \equiv \sum_1 7_1 \cdot 10^1$$

- Ejemplo:

$$3278,52 = 3 \cdot 10^3 + 2 \cdot 10^2 + 7 \cdot 10^1$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- El valor del número se calcula mediante el polinomio:

$$N \equiv ... + n_3 \cdot b_3 + n_2 \cdot b_2 + n_1 \cdot b_1 + n_0 \cdot b_0 + n_{-1} \cdot b_{-1} ...$$

$$N \equiv \sum_0 8_0 \cdot 10^0$$

- Ejemplo:

$$3278,52 = 3 \cdot 10^3 + 2 \cdot 10^2 + 7 \cdot 10^1 + 8 \cdot 10^0$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- El valor del número se calcula mediante el polinomio:

$$N \equiv ... + n_3 \cdot b_3 + n_2 \cdot b_2 + n_1 \cdot b_1 + n_0 \cdot b_0 + n_{-1} \cdot b_{-1} ...$$

$$N \equiv \sum_{-1} 5_{-1} \cdot 10^{-1}$$

- Ejemplo:

$$3278,52 = 3 \cdot 10^3 + 2 \cdot 10^2 + 7 \cdot 10^1 + 8 \cdot 10^0 + 5 \cdot 10^{-1}$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- El valor del número se calcula mediante el polinomio:

$$N \equiv ... + n_3 \cdot b_3 + n_2 \cdot b_2 + n_1 \cdot b_1 + n_0 \cdot b_0 + n_{-1} \cdot b_{-1} ...$$

$$N \equiv \sum_{-2} 2_{-2} \cdot 10^{-2}$$

- Ejemplo:

$$3278,52 = 3 \cdot 10^3 + 2 \cdot 10^2 + 7 \cdot 10^1 + 8 \cdot 10^0 + 5 \cdot 10^{-1} \\ + 2 \cdot 10^{-2}$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- El valor del número se calcula mediante el polinomio:

$$N \equiv ... + n_3 \cdot b_3 + n_2 \cdot b_2 + n_1 \cdot b_1 + n_0 \cdot b_0 + n_{-1} \cdot b_{-1} ...$$

$$N \equiv \sum_{-2} 2_{-2} \cdot 10^{-2}$$

- Ejemplo:

$$\begin{aligned} 3278,52 &= 3 \cdot 10^3 + 2 \cdot 10^2 + 7 \cdot 10^1 + 8 \cdot 10^0 + 5 \cdot 10^{-1} \\ &\quad + 2 \cdot 10^{-2} = \mathbf{3278,52}_{(10)} \end{aligned}$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

Conversión decimal - base b

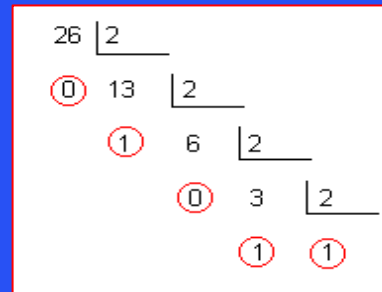
- Método de divisiones sucesivas entre la base b
- Para números fraccionarios se realizan multiplicaciones sucesivas por la base b .
- Consideración de restos mayores que 9 y Error de truncamiento

Ejemplos:

$$26_{(10)} = 11010_{(2)}$$

$$0,1875_{(10)} = 0,0011_{(2)}$$

$$26,1875_{(10)} = 11010,0011_{(2)}$$



$\begin{array}{r} 0,1875 \\ \times 2 \\ \hline 0,3750 \end{array}$	$\begin{array}{r} 0,3750 \\ \times 2 \\ \hline 0,7500 \end{array}$	$\begin{array}{r} 0,7500 \\ \times 2 \\ \hline 1,5000 \end{array}$	$\begin{array}{r} 0,5000 \\ \times 2 \\ \hline 1,0000 \end{array}$
--	--	--	--

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- Rango de representación: Conjunto de valores representable. Con n cifras en la base b podemos formar b^n combinaciones distintas. $[0..b_{n-1}]$
- Sistema de numeración en base dos o binario

$b = 2$ (binario) $\{0,1\}$

$n = 3 \Rightarrow 2^3 = 8$ Combinaciones

de números binarios del 0 al 7

Decimal	Binario
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

- Ejemplos:

$$110100_{(2)} = (1 \cdot 2^5) + (1 \cdot 2^4) + (1 \cdot 2^2) = 25 + 24 + 22 = 32 + 16 + 4 = 52_{10}$$

$$0,10100_{(2)} = ?$$

$$10100,001_{(2)} = ?$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- Rango de representación: Conjunto de valores representable. Con n cifras en la base b podemos formar b^n combinaciones distintas. $[0..b_{n-1}]$
- Sistema de numeración en base dos o binario

$b = 2$ (binario) $\{0,1\}$

$n = 3 \Rightarrow 2^3 = 8$ Combinaciones

de números binarios del 0 al 7

Decimal	Binario
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

- Ejemplos:

$$110100_{(2)} = (1 \cdot 2^5) + (1 \cdot 2^4) + (1 \cdot 2^2) = 25 + 24 + 22 = 32 + 16 + 4 = 52_{10}$$

$$0,10100_{(2)} = 2^{-1} + 2^{-3} = (1/2) + (1/8) = 0,625$$

$$10100,001_{(2)} = ?$$

1. Codificación de la Información Numérica

Sistemas de numeración y cambio de base

- Rango de representación: Conjunto de valores representable. Con n cifras en la base b podemos formar b^n combinaciones distintas. $[0..b_{n-1}]$
- Sistema de numeración en base dos o binario

$b = 2$ (binario) $\{0,1\}$

$n = 3 \Rightarrow 2^3 = 8$ Combinaciones

de números binarios del 0 al 7

Decimal	Binario
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

- Ejemplos:

$$110100_2 = (1 \cdot 2^5) + (1 \cdot 2^4) + (1 \cdot 2^2) = 25 + 24 + 22 = 32 + 16 + 4 = 5210$$

$$0,10100_2 = 2^{-1} + 2^{-3} = (1/2) + (1/8) = 0,625$$

$$10100,001_2 = 2^4 + 2^2 + 2^{-3} = 16 + 4 + (1/8) = 20,12510$$

1. Codificación de la Información Numérica

Aritmética binaria

- Operaciones básicas

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	0 (1)

A	B	A*B
0	0	0
0	1	0
1	0	0
1	1	1

A	B	A - B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

A	B	A/B
0	0	--
0	1	0
1	0	--
1	1	1

1. Codificación de la Información Numérica

Aritmética binaria

■ Ejemplos

■ Suma y Resta

$$\begin{array}{r} + \quad 1110101 \\ \quad 1110110 \\ \hline 11101011 \end{array} \qquad \begin{array}{r} 1101010 \\ - \quad 1010111 \\ \hline 0010011 \end{array}$$

■ Multiplicación

$$\begin{array}{r} 1101010 \\ \times 11 \\ \hline 1101010 \\ 1101010 \\ \hline 10011110 \end{array} \qquad \begin{array}{r} 1010011 \\ \times 10 \\ \hline 0000000 \\ 1010011 \\ \hline 10100110 \end{array}$$

■ División

$$\begin{array}{r} 1101.011 \quad \overline{)101} \\ -101 \\ \hline 00110 \\ -101 \\ \hline 00111 \\ -101 \\ \hline 10 \end{array}$$

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - \\ 1010111 \\ \hline \end{array}$$

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 1010111 \\ \hline \end{array}$$

Diagram illustrating binary subtraction. The minuend is 1101010 and the subtrahend is 1010111. A borrow of 1 is indicated by a red '1' between the two numbers at the 2nd bit from the right. A purple box highlights the borrow '1' and the result '1' in the 1st bit position.

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

Diagram illustrating the truth table for binary subtraction. The table shows the result A-B for all combinations of A and B. A purple box highlights the row where A=0 and B=1, showing the result 1 (1), indicating a borrow.

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 1010111 \\ \hline \end{array}$$

Diagram illustrating binary subtraction. The minuend is 1101010 and the subtrahend is 1010111. A borrow of 1 is indicated by a red '1' between the two numbers at the 3rd and 4th positions from the right. The result of the subtraction is 0010011, with the final two digits '11' shown below the line.

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 1010111 \\ \hline 011 \end{array}$$

Diagram illustrating binary subtraction. The minuend is 1101010 and the subtrahend is 1010111. A borrow of 1 is shown in red above the third column from the right. The result is 011.

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 1010111 \\ \hline 0011 \end{array}$$

Diagram illustrating binary subtraction. The minuend is 1101010 and the subtrahend is 1010111. A borrow of 1 is taken from the 4th bit of the minuend (the first 1 from the right), which is crossed out and replaced by 0. This borrow is represented by a red '1' placed between the 4th and 5th bits of the minuend. The resulting difference is 0011.

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 111111 \\ \hline 10011 \end{array}$$

Diagram illustrating binary subtraction. The minuend is 1101010 and the subtrahend is 111111. The result is 10011. A vertical box highlights the borrowing process: a '1' is borrowed from the third bit of the minuend (0) and placed between the second and third bits of the subtrahend (111111), effectively changing the subtrahend to 1111111 for the purpose of the subtraction in that column.

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

Table illustrating the binary subtraction operation A - B. The result for 0 - 1 is 1 (1), indicating a borrow of 1 from the next higher bit.

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 1010111 \\ \hline 010011 \end{array}$$

Diagram illustrating binary subtraction. The minuend is 1101010 and the subtrahend is 1010111. The result is 010011. A borrow of 1 is shown in red at the second bit position, and the resulting 0 is shown in a box at the first bit position.

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

Table showing the binary subtraction results for A, B, and A-B. The first row (0, 0, 0) is highlighted with a box.

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del $0 - 1$.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 1111 \\ \hline 0010011 \end{array}$$

Diagram illustrating binary subtraction. The minuend is 1101010 and the subtrahend is 1111. The result is 0010011. A borrow of 1 is shown in red at the second position from the right, and the first bit of the result (0) is highlighted with a blue box.

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

Truth table for binary subtraction (A - B). The first row (0 - 0 = 0) is highlighted with a blue box.

1. Codificación de la Información Numérica

Aritmética binaria

- La Resta
- Basándonos en la tabla de la Resta, el único caso en que se produce un borrow es el del 0 – 1.
- Para simplificar el procedimiento, se propone colocar el borrow entre el minuendo y sustraendo para luego operarlo en el siguiente término:
- Ejemplo:

$$\begin{array}{r} 1101010 \\ - 1010111 \\ \hline 0010011 \end{array}$$

A	B	A-B
0	0	0
0	1	1 (1)
1	0	1
1	1	0

1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

■ Octal

- $b = 8$ (octal) $\{0,1,2,3,4,5,6,7\}$
- Correspondencia con el binario

$8 = 2^3 \Rightarrow$ Una cifra en octal corresponde a 3 binarias

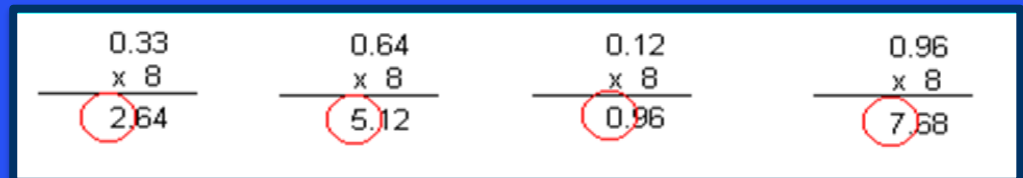
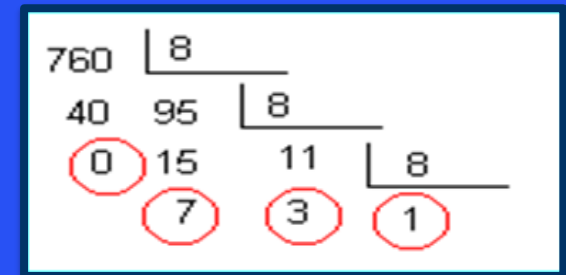
■ Ejemplos

$$10001101100.11010_{(2)} = 2154.64_{(8)}$$

$$537.24_{(8)} = 101011111.010100_{(2)}$$

Conversión Decimal - Octal

$$760.33_{(10)} \cong 1370.2507_{(8)}$$



1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

■ Octal

$b = 8$

$\{0,1,2,3,4,5,6,7\}$

■ Hexadecimal

$b = 16$

$\{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$

Hexa	Decimal	Octal	Binario
0	0	0	0
1	1	1	1
2	2	2	10
3	3	3	11
4	4	4	100
5	5	5	101
6	6	6	110
7	7	7	111
8	8	10	1000
9	9	11	1001
A	10	12	1010
B	11	13	1011
C	12	14	1100
D	13	15	1101
E	14	16	1110
F	15	17	1111

1. Codificación de la Información Numérica

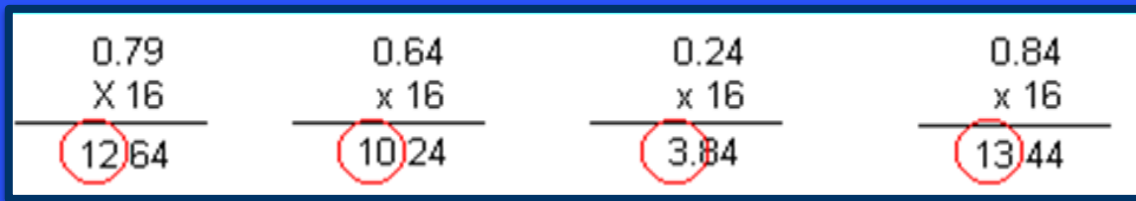
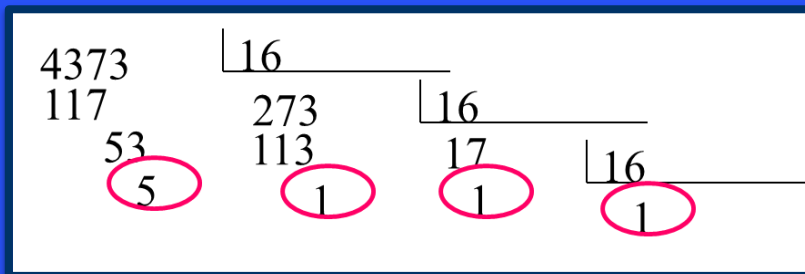
Sistemas de representación y codificación de los números

■ Ejemplos

$10010111011111.10111012 = 25DF.BAH$

■ Conversión Decimal – Hexadecimal

$4373.7910 \cong 1115.CA3D16$



1. Codificación de la Información Numérica

Resumen de Conversión de Sistemas de Numeración

ORIGEN DESTINO	Decimal	Binario	Octal	Hexadecimal
Decimal		Divisiones sucesivas por la base(2)	Divisiones sucesivas por la base (3)	Divisiones sucesivas por la base (16)
Binario	Multiplicación por potencias de la base (2)		Agrupar de a 3 bits desde la derecha	Agrupar de a 4 bits desde la derecha
Octal	Multiplicación por potencias de la base (8)	Expresar cada cifra octal por su binario de 3 bits equivalente		Convertir octal a binario y luego éste a Hexadecimal
Hexadecimal	Multiplicación por potencias de la base (16)	Expresar cada cifra Hexa por su binario de 4 bits equivalente	Convertir Hexa a binario y luego éste a Octal	

1. Codificación de la Información Numérica

Complemento a la base menos uno

- Los valores positivos se representan en formato SM (Signo y Magnitud).
- Los valores negativos se obtienen restando la magnitud del número a la base menos uno.
- Convierte las restas en sumas.
- Doble representación del 0.
- Ejemplos **Base 10**

$$n = 3 \quad -63_{10} = 936_{C9} \Rightarrow 936 = 999 - 63$$

$$-16_{10} = 983_{C9} \Rightarrow 983 = 999 - 16$$

$$n = 4 \quad -16_{10} = 9983_{C9} \Rightarrow 9983 = 9999 - 16$$

$$\begin{array}{r} \text{Operación: } 77 - 63 \\ 77 \\ -63 \\ \hline 14 \end{array}$$

$$\begin{array}{r} + \quad 077_{C9} \\ \quad 936_{C9} \\ \hline (1)013 \\ + \quad \text{└─} 1 \\ \hline \quad 014_{C9} \end{array}$$

1. Codificación de la Información Numérica

Complemento a la base

- Se intercambian ceros por unos y unos por ceros
- Rango : $[-2^{n-1} + 1, 2^{n-1} - 1]$
- Ejemplos: $n = 6$ C_1 de $-10010_2 = 101101_{C_1}$
 C_1 de $-100111_2 = \text{no representable}$
 C_1 de $0 = \{000000_{C_1}, 111111_{C_1}\}$

$$\begin{array}{r} 111111 \\ - 010010 \\ \hline 101101 \end{array}$$

Operación: $1000111_2 - 10010_2$

Restando en binario natural

$$\begin{array}{r} 1000111_2 \\ - 0010010_2 \\ \hline 0110101_2 \end{array}$$

Sumando en C1 (n=8)c

$$\begin{array}{r}
 01000111_{C1} \\
 + \quad 11101101_{C1} \\
 \hline
 (1)00110100 \\
 + \quad \quad \quad \rightarrow 1 \\
 \hline
 00110101_{C1}
 \end{array}$$

Código Gray

Sistemas de representación y codificación de los números

- Código Gray
 - Código no ponderado, continuo y cíclico
 - Basado en un sistema binario
 - Dos números sucesivos sólo varían en un bit
 - Usado en la Segunda generación ya que las válvulas generaban picos de ruido debido a que trabajaban con altos voltajes, permitiendo la detección de errores.
 - En la tercera y cuarta generación (transistores y CI) se dejó de usar para ese fin.
 - En la actualidad se emplea como parte del algoritmo de diseño de los mapas de Karnaugh para diseño de circuitos combinacionales y secuenciales.

Código Gray

Sistemas de representación y codificación de los números

■ Código Gray

- Código no ponderado, continuo y cíclico
- Basado en un sistema binario
- Dos números sucesivos sólo varían en un bit

2 bits	3 bits	4 bits	Decimal
0 0	0 0 0	0 0 0 0	0
0 1	0 0 1	0 0 0 1	1
1 1	0 1 1	0 0 1 1	2
1 0	0 1 0	0 0 1 0	3
	1 1 0	0 1 1 0	4
	1 1 1	0 1 1 1	5
	1 0 1	0 1 0 1	6
	1 0 0	0 1 0 0	7
		1 1 0 0	8
		1 1 0 1	9
		1 1 1 1	10
		1 1 1 0	11
		1 0 1 0	12
		1 0 1 1	13
		1 0 0 1	14
		1 0 0 0	15

Código Gray

Sistemas de representación y codificación de los números

■ **Conversión Binario - Gray**

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$

Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$

1 $\xrightarrow{+}$ 1 $\xrightarrow{+}$ 0 $\xrightarrow{+}$ 0 $\xrightarrow{+}$ 0 $\xrightarrow{+}$ 1 $\xrightarrow{+}$ 1 $\xrightarrow{+}$ 0

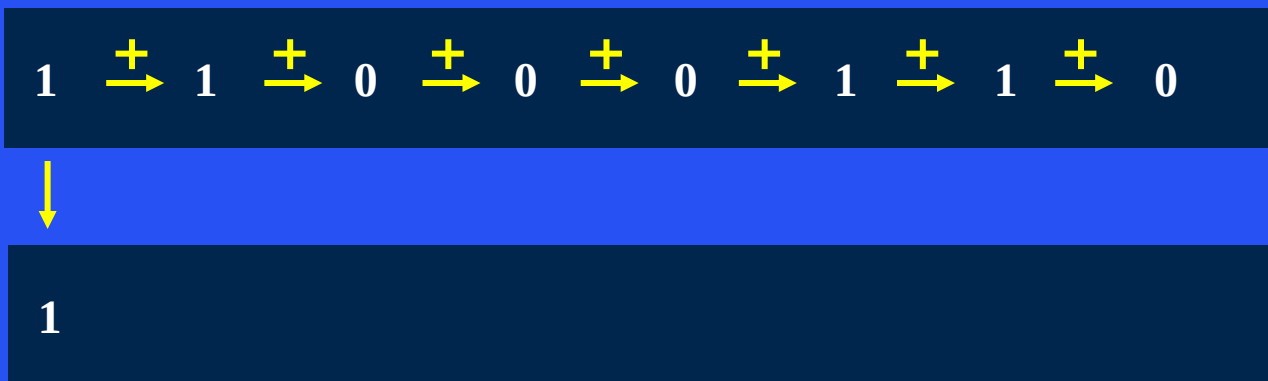
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



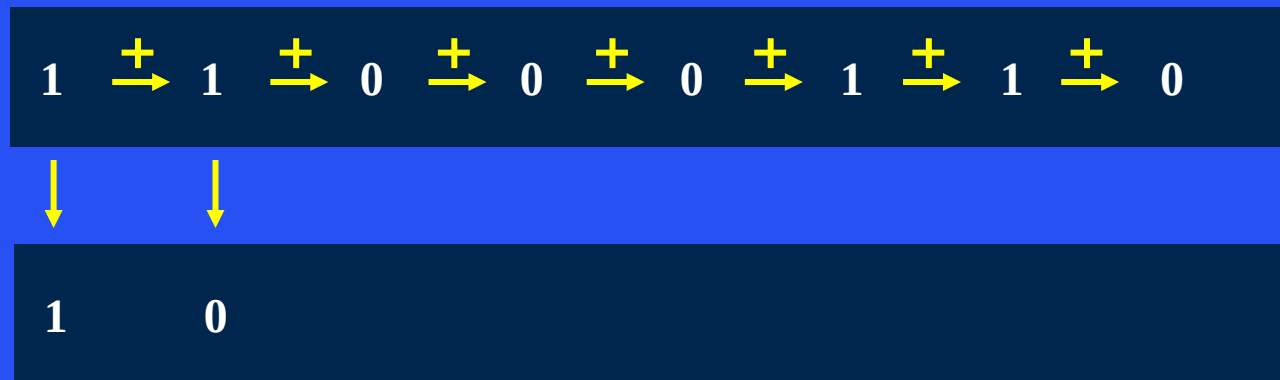
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



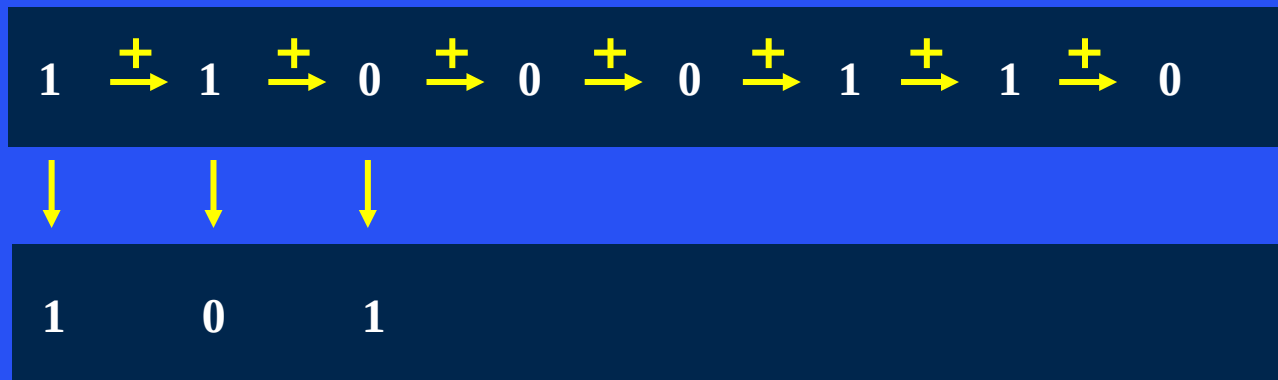
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



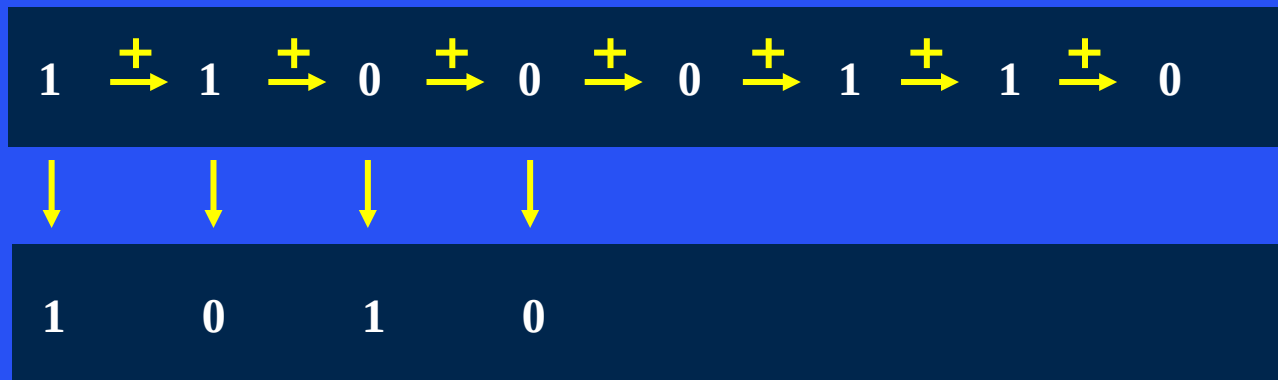
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



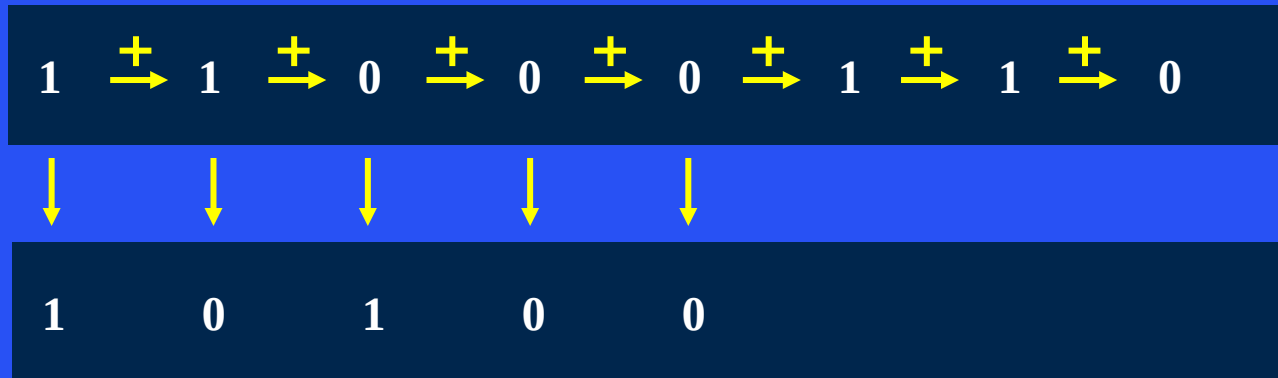
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



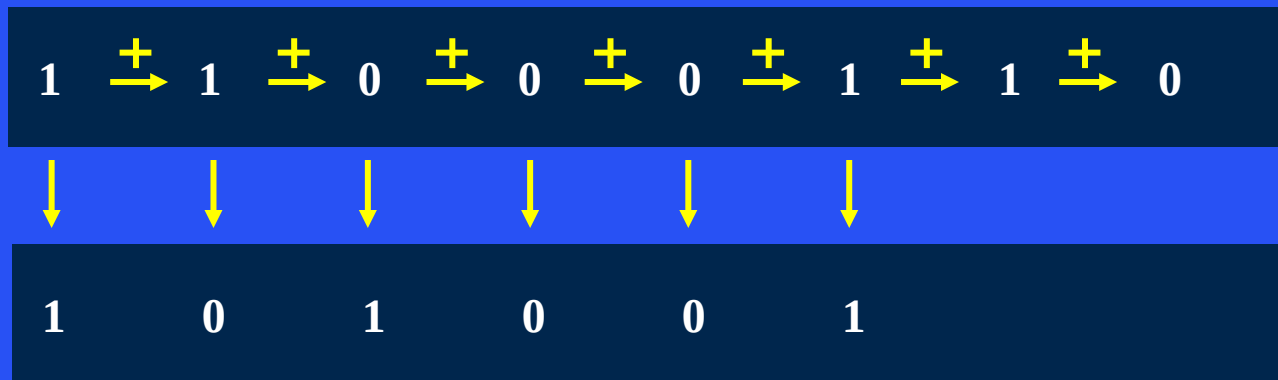
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



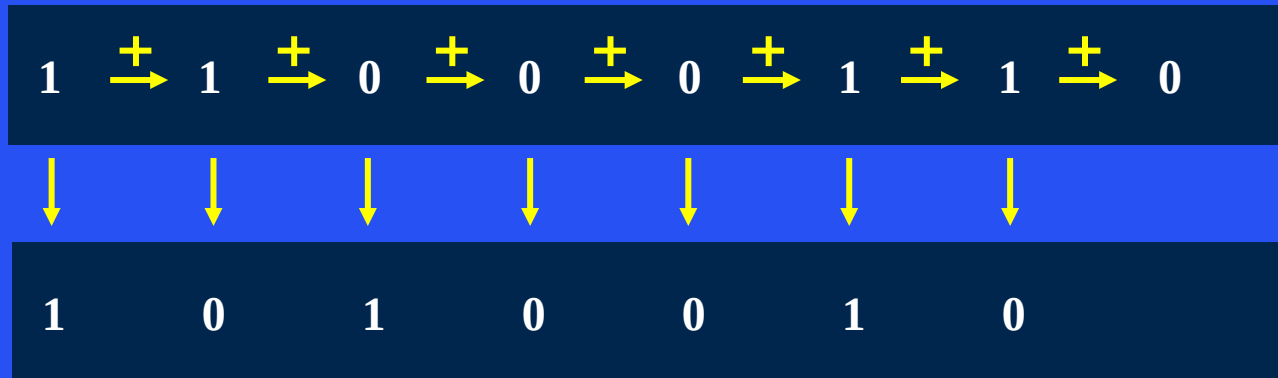
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



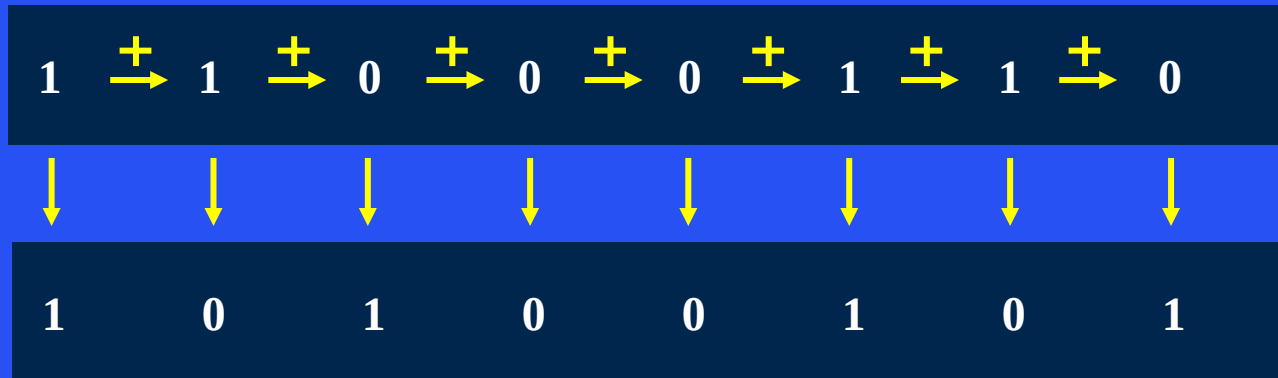
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (\quad)_{\text{gray}}$



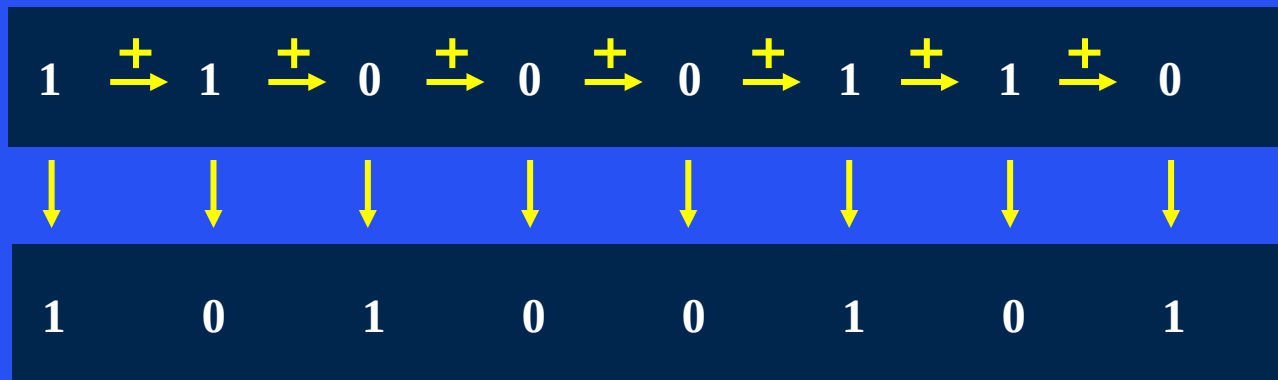
Código Gray

Sistemas de representación y codificación de los números

■ Conversión Binario - Gray

A partir del primer bit sumamos el bit binario que queremos obtener con el de su izquierda (se descarta el acarreo)

Ejemplo: $(1\ 1\ 0\ 0\ 0\ 1\ 1\ 0)_2 = (1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}}$



Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(10100101)_{\text{gray}} = (\quad)_2$

1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

1 0 1 0 0 1 0 1



1

1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

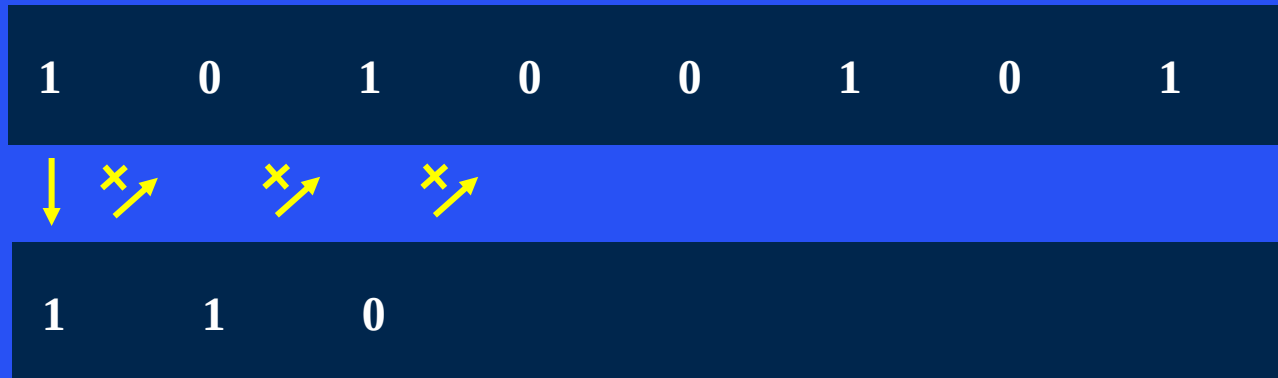


1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

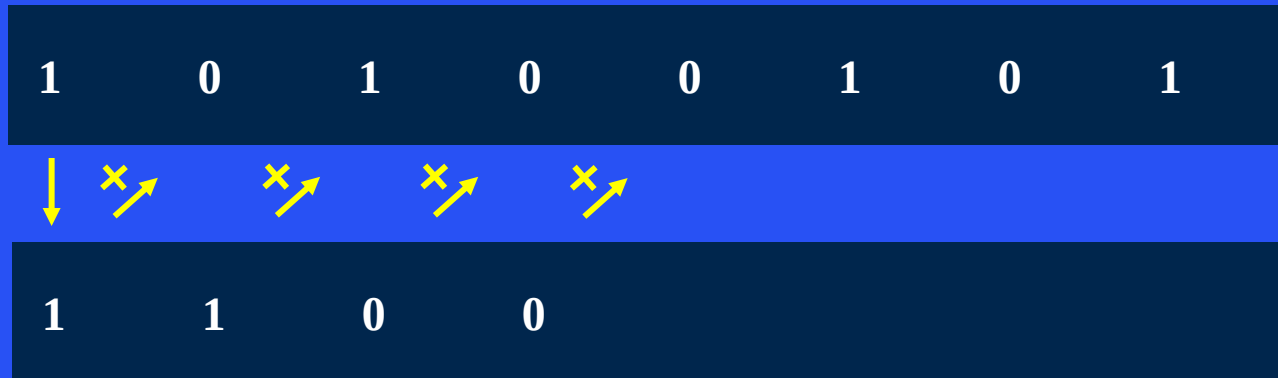


1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

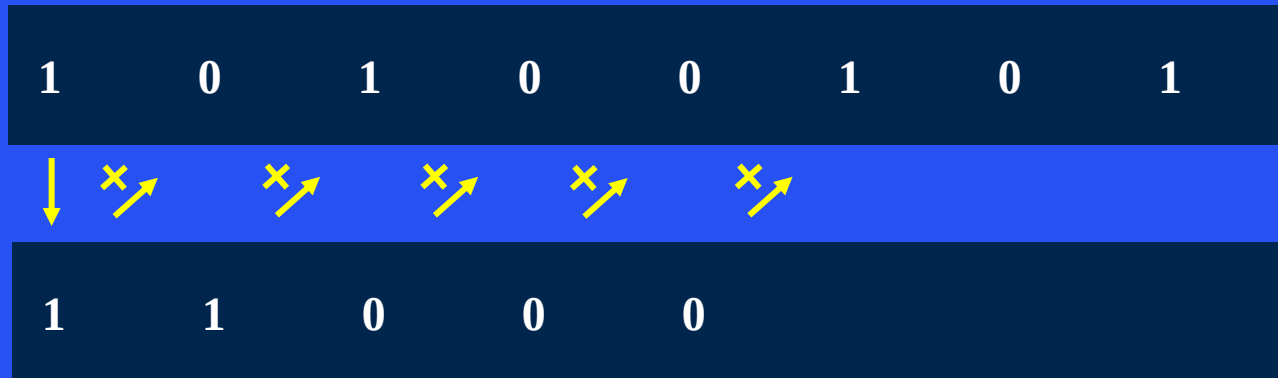


1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

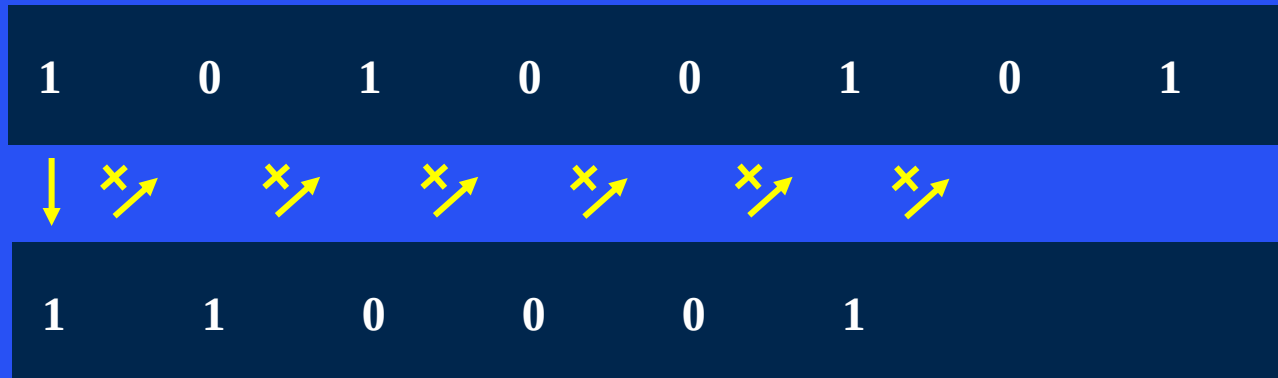


1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

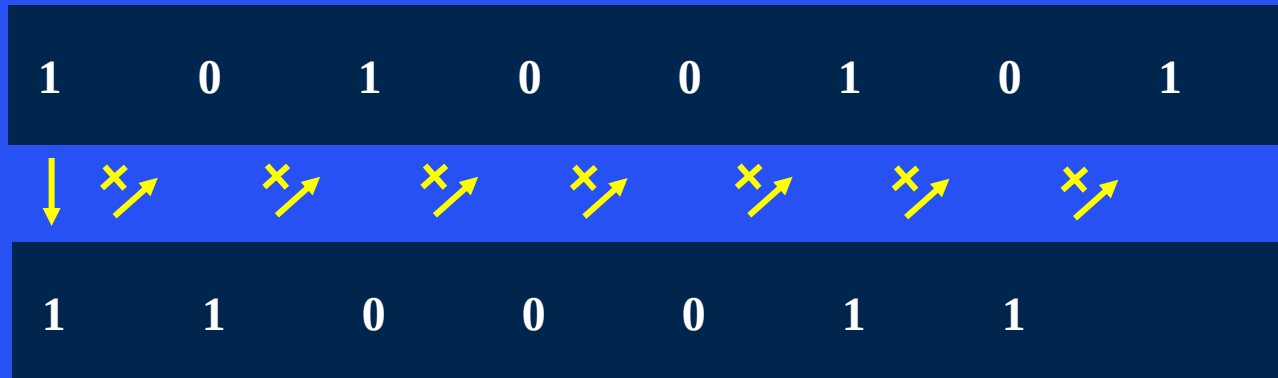


1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

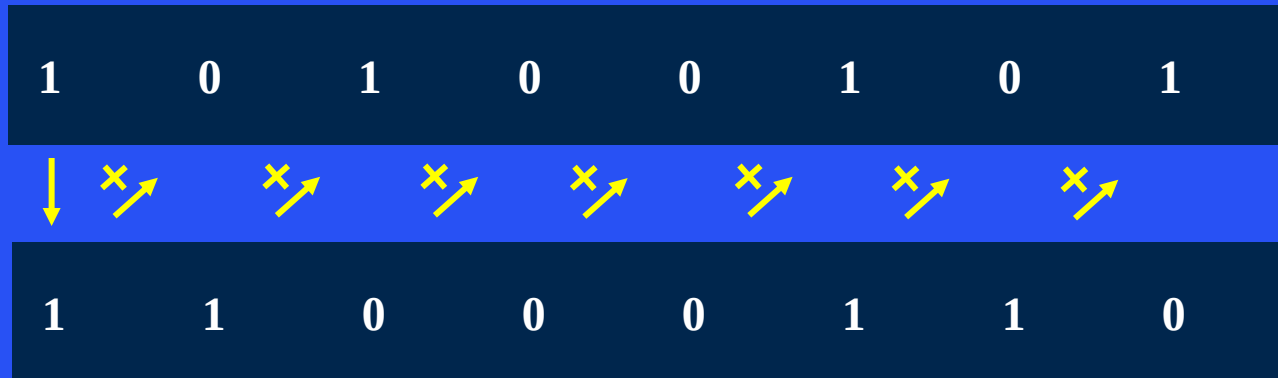


1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(1\ 0\ 1\ 0\ 0\ 1\ 0\ 1)_{\text{gray}} = (\quad)_2$

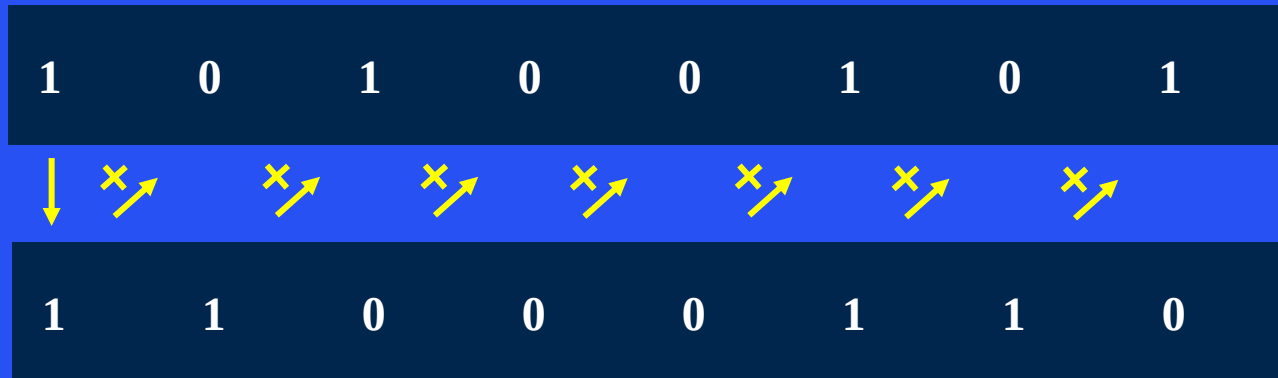


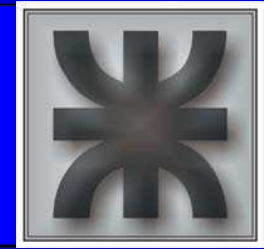
1. Codificación de la Información Numérica

Sistemas de representación y codificación de los números

- **Conversión Gray – Binario:** Se baja el bit mas significativo y se le suma el código gray adyacente, descartando el acarreo.

Ejemplo: $(10100101)_{\text{gray}} = (11000110)_2$





[Arquitectura de Computadoras]

REPRESENTACIÓN DE LA INFORMACIÓN

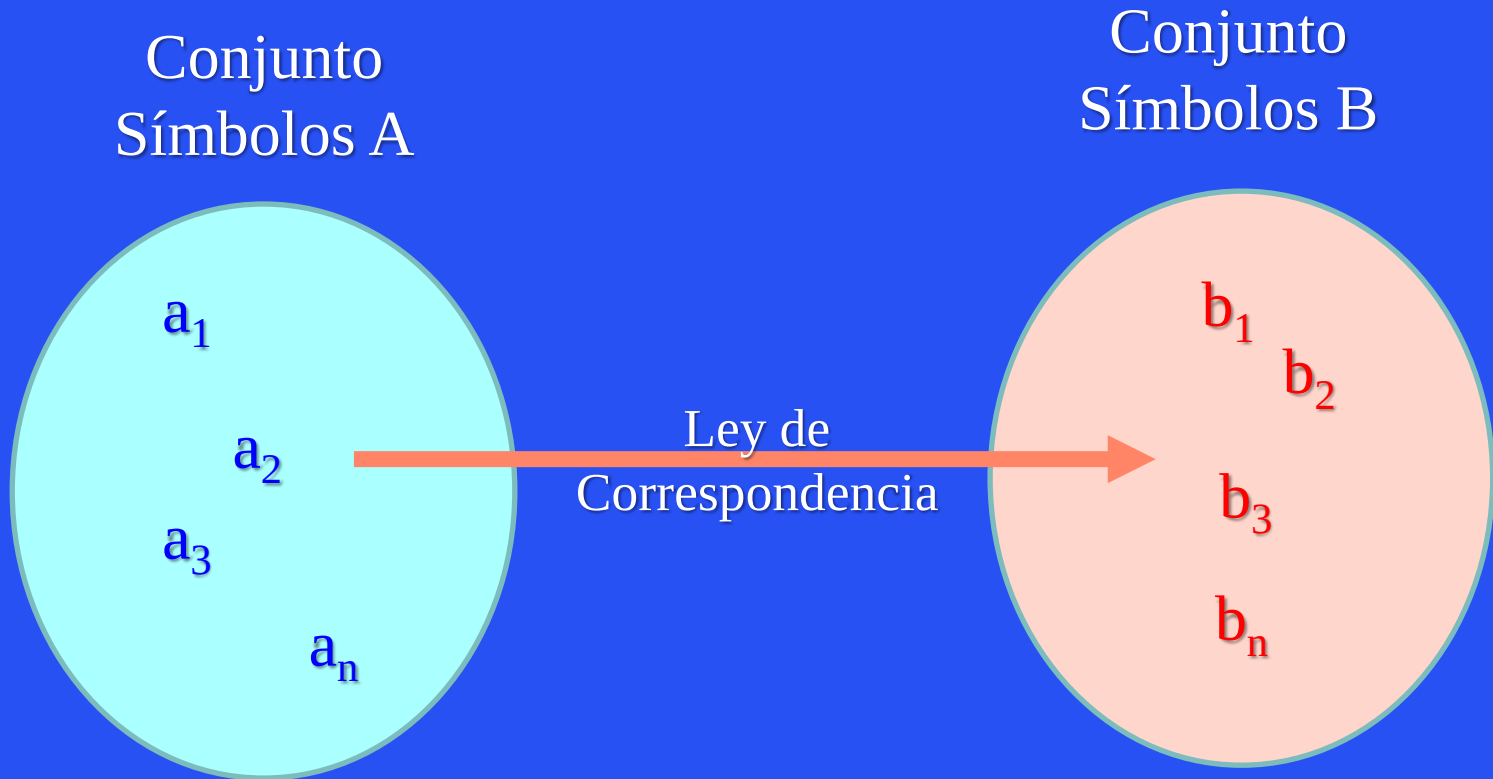
Ing. Gustavo Maurokefalidis

Representación de la Información

CONTENIDO

- Codificación de la Información numérica
- Códigos de representación de caracteres alfanuméricos
- Códigos de representación decimal (bcd)
 - BCD Puro o Natural (8421)
 - BCD Empaquetado y Desempaquetado
 - BCD Exceso en Tres
 - AIKEN (2421)
- Códigos Redundantes
 - Paridad Par e Impar
 - Código Entrelazado
 - Código de Hamming

Codificación de la Información



Un sistema de Codificación establece una Ley de correspondencia entre un conjunto de símbolos de un conjunto A y otro conjunto de símbolos de un conjunto B, de manera que pueda pasarse de un mensaje codificado en uno de ellos pueda ser convertido al otro respetando las reglas impuestas por dicha ley

Códigos de representación de caracteres alfanuméricos

Un **código alfanumérico** establece la relación necesaria para que una computadora digital que procesa solamente dígitos binarios interprete el lenguaje que utiliza el usuario (caracteres numéricos, alfabéticos o especiales).

Por ejemplo, con 7 bits se obtienen $2^7 = 128$ combinaciones distintas, que son suficientes para asignar un símbolo alfabético, numérico o especial a cada una de ellas.

Supóngase que se ingresa el dato alfanumérico “+105”, el proceso de codificación hará que éste quede registrado en la memoria, por ejemplo, en código ASCII, como se muestra a continuación.

Registros de memoria	0010 1011	+
	0011 0001	1
	0011 0000	0
	0011 0101	5

Códigos de representación de caracteres alfanuméricos

Si bien una computadora es una “**calculadora aritmética**” por excelencia, gran parte de sus aplicaciones consiste en el tratamiento de cadenas de caracteres llamadas “**strings**”.

El **código ASCII** (American Standard Code for Information Interchange – código estándar americano para intercambio de información–) representa cada carácter con 7 bits (o sea que en este caso la medida del byte es de 7 bits), que permite determinar $2^7 = 128$ combinaciones distintas, suficientes para establecer una única relación carácter-combinación binaria, que se representa en la tabla.

Códigos de representación de caracteres alfanuméricos

EBCDIC (Extended Binary Coded Decimal Interchange Code)

Bits 3210	7654 0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	NUL	DLE	DS	IRS	SP	&	-	e	O	*	μ	c	ä	â	E	0
0001	SOH	DC1	SOS	ITB	RSP	`	/	\	a	j	ü	f	A	J	÷	1
0010	STX	DC2	FS	SYN	â	ë	A	E	b	k	s	#	B	K	S	2
0011	ETX	TM	WUS	IR	{	ë	#	E	c	l	t	-	C	L	T	3
0100	PF	RES	BYP	PN	ä	ë	A	E	d	m	u	©	D	M	U	4
0101	HT	NL	LF	RS	ä	i	A	I	e	n	v	[	E	N	V	5
0110	LC	BS	ETB	UC	ä	i	A	I	f	o	w	¶	F	O	W	6
0111	DEL	IL	ESC	EOT	}	I	\$	I	g	p	x	¼	G	P	X	7
1000	GE	CAN	SA	SBS	ç	i	Ç	I	h	q	y	¼	H	Q	Y	8
1001	SPS	EM	SFE	IT	ñ	ß	N	ë	i	r	z	¼	I	R	Z	9
1010	SMM	CC	SM	RFF	§	≈	ö	:	α	*	i	-	SHY	'	²	²
1011	VT	CU1	CU2	CU3	.	Ä	,	Ä	»	*	¿		ö	û	O	U
1100	FF	IFS	MFA	DC4	<	*	%	O	ö	≈	Ð	—	ï	~	@	U
1101	CR	IGS	ENQ	NAK	(	)	-	°	ý	,	Y	]	ö	û	O	U
1110	SO	IRS	ACK		+	:	>	=	þ	Æ	þ	·	ö	û	O	U
1111	SI	IUS	BEL	SUB	!	^	?	"	±	]	©	×	ö	ý	O	EO

M	a	r	t	e	s		1	0
---	---	---	---	---	---	--	---	---

8	7	6	5	4	3	2	1	0
1101 0100	1000 0001	1001 1001	0011 1010	0101 0111	0010 1010	0000 0100	1111 0001	1111 0000

Códigos de representación de caracteres alfanuméricos

ZONA

ASCII (American Standard Code for Information Interchange)

HEX. N°	HEX. N°	0	1	2	3	4	5	6	7
HEX. N°	BITS 654 3210	000	001	010	011	100	101	110	111
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	\$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	:	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US						

DIGITO

SP (Espacio) = 010 0000
Zona Dígito

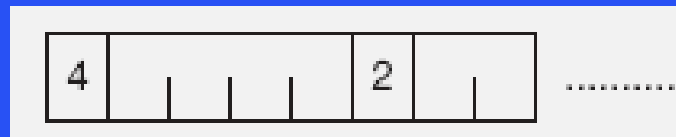
Códigos de representación de caracteres alfanuméricos

ASCII

Delimitación de strings

Las cadenas de caracteres alfanuméricas (strings), procesadas y almacenadas en la computadora, pueden ser:

- **De longitud fija:** cada dato ocupa un número determinado de bytes fijo. Por lo tanto, el comienzo y el final de cada dato puede conocerse con rapidez.
- **De longitud variable:** Cada dato tiene en su inicio un campo en el que se indica la longitud en bytes, como lo muestra a continuación.



UNICODE

- Diseñado con la intención de superar las limitaciones presentes en todas las codificaciones de texto diseñadas hasta ese momento
- El estándar permite la codificación de textos mundial fiable y viable.
- Podría verse como un " ASCII de cuerpo ancho " que se ha ampliado a 16 bits para abarcar los caracteres de todos los idiomas vivos del mundo.
- El Consorcio Unicode se constituyó en California el 3 de enero de 1991 y el primer volumen del Estándar Unicode se publicó en octubre de ese mismo año. El segundo volumen, que ahora incorpora los ideogramas Han, se publicó en junio de 1992.

Códigos de representación de caracteres alfanuméricos

UNICODE

P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
0050	0051	0052	0053	0054	0055	0056	0057	0058	0059	005A	005B	005C	005D	005E	005F
`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
0060	0061	0062	0063	0064	0065	0066	0067	0068	0069	006A	006B	006C	006D	006E	006F
p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
0070	0071	0072	0073	0074	0075	0076	0077	0078	0079	007A	007B	007C	007D	007E	007F
PAD	HOP	BPH	NBH	IND	NEL	SSA	ESA	HTS	HTJ	LTS	PLD	PLU	RI	SS2	SS3
0080	0081	0082	0083	0084	0085	0086	0087	0088	0089	008A	008B	008C	008D	008E	008F
DCS	PU1	PU2	STS	CCH	MW	SPA	EPA	SOS	SG CI	SCI	CSI	ST	OSC	PM	APC
0090	0091	0092	0093	0094	0095	0096	0097	0098	0099	009A	009B	009C	009D	009E	009F
¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬		®	¯	
00A0	00A1	00A2	00A3	00A4	00A5	00A6	00A7	00A8	00A9	00AA	00AB	00AC	00AD	00AE	00AF
°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
00B0	00B1	00B2	00B3	00B4	00B5	00B6	00B7	00B8	00B9	00BA	00BB	00BC	00BD	00BE	00BF
À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
00C0	00C1	00C2	00C3	00C4	00C5	00C6	00C7	00C8	00C9	00CA	00CB	00CC	00CD	00CE	00CF
Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
00D0	00D1	00D2	00D3	00D4	00D5	00D6	00D7	00D8	00D9	00DA	00DB	00DC	00DD	00DE	00DF

↑ Volver arriba

Q Buscar bloques Unicode

0: Plano básico multilingüe

Latín básico 0000–007F

Latín-1 suplemento 0080–00FF

Latín extendido A 0100–017F

Latín extendido B 0180–024F

Extensiones del AFI 0250–02AF

Separación modificador letras 02B0–02FF

Diacríticos (combinables) 0300–036F

Griego y copto 0370–03FF

Cirílico 0400–04FF

Cirílico suplemento 0500–052F

Armenio 0530–058F

Hebreo 0590–05FF

Árabe 0600–06FF

Siriaco 0700–074F

Árabe suplemento 0750–077F

Thaana 0780–07BF

NKO 07C0–07FF

Samaritano 0800–083F

Código de Representación Decimal BCD

- El Código **BCD NATURAL O PURO**: permiten la representación de números decimales (0 a 9) en bloques binarios de 4 bits.
- Se denomina así porque los pesos en cada bloque coinciden con el valor de los 4 primeros pesos del sistema binario puro. Ésta es la razón por la que al BCD puro se lo suele llamar BCD 8421. En la siguiente tabla se encuentran las equivalencias entre decimal y BCD puro.

DECIMAL	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Código de Representación Decimal BCD

Los códigos BCD tienen dos aplicaciones en el campo de la computación

- **DESEMPAQUETADO:** Los números se codifican utilizando un Byte por dígito (zona y dígito) y es utilizado para transmisión desde hacia periféricos del computador
- **EMPAQUETADO:** Para el tratamiento de cálculos numéricos la ALU no necesita la ZONA, por lo que la elimina, reduciendo la codificación a solo 4 Bits por dígito rellendo con unos los de orden superior. Se los utiliza para cálculos en el entorno de la ALU. El signo se lo incluye por lo general en la parte sobrante del octeto (cabecera o cola del número) siendo **F o C (positivo)**, **B o D (negativo)**. Se agrega un cero adelante para mantener la paridad de la cadena en caso de ser necesaria.

Código de Representación Decimal BCD

BCD (Empaquetado y Desempaquetado)

La misma UAL que opera en binario puro puede operar en BCD. Lo que cambia es la forma en que se producen las distintas etapas para llevar a cabo la operación. Estas etapas están definidas por instrucciones en el programa, que permiten la gestión de la operación, considerando como base del sistema la decimal y no la binaria.

	+3456	+279	-3456	-279
Desempaquetado	2B 33 34 35 36	2B 32 37 39	2D 33 34 35 36	2D F2 F7 F9
Empaquetado	03456 B	279 B	03456 D	279 D

Código de Representación Decimal BCD

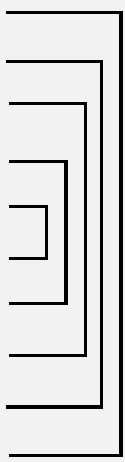
BCD EXCESO EN TRES

- Es un código decimal codificado en binario autocomplementario y un sistema de numeración.
- Utilizado en varias computadoras antiguas al igual que en cajas registradoras y calculadoras electrónicas portátiles en la década del '70
- Se obtiene a partir del BCD Puro sumándole 3 a cada cifra decimal.
- Con esto se logra un código simétrico o auto complementario, facilitando la operación de hallar el complemento restringido de un valor binario, solo con invertir sus dígitos.

Código de Representación Decimal BCD

BCD EXCESO EN TRES

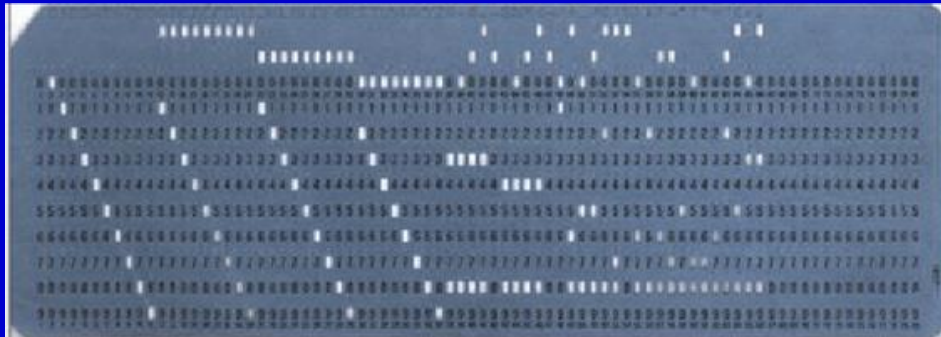
En la siguiente figura se muestran las equivalencias entre decimal y BCD exceso tres. Obsérvese que 0111 (4) es el complemento restringido de 1000 (5), o que 0011 (0) es el complemento restringido de 1100 (9).

<i>Valor decimal</i>	<i>BCD exceso tres</i>	
0	0011	 "Código autocomplementario"
1	0100	
2	0101	
3	0110	
4	0111	
5	1000	
6	1001	
7	1010	
8	1011	
9	1100	

Código de Representación Decimal BCD

CÓDIGO 5421

- Surge de sumar los unos en las posiciones 5421 para obtener su equivalente decimal
- Del 0 al 4 se asigna un cero al bit de mayor peso
- Del 5 al 9 se asigna un 1 al bit de mayor peso
- El resto de los bits son iguales en un grupo y en otro
- Fue muy usado en la época de las tarjetas perforadas (2° Generación) para ahorrar espacio en la representación de números.



Código de Representación Decimal BCD

Sistemas de representación y codificación de los números

■ Código BCD - *Binary Coded Decimal*

Posee dígitos decimales codificados en binario

D e c i m a l	B C D n a t u r a l	B C D e x c e s o 3	B C D A i k e n	B C D 5 4 2 1
0	0 0 0 0	0 0 1 1	0 0 0 0	0 0 0 0
1	0 0 0 1	0 1 0 0	0 0 0 1	0 0 0 1
2	0 0 1 0	0 1 0 1	0 0 1 0	0 0 1 0
3	0 0 1 1	0 1 1 0	0 0 1 1	0 0 1 1
4	0 1 0 0	0 1 1 1	0 1 0 0	0 1 0 0
5	0 1 0 1	1 0 0 0	1 0 1 1	1 0 0 0
6	0 1 1 0	1 0 0 1	1 1 0 0	1 0 0 1
7	0 1 1 1	1 0 1 0	1 1 0 1	1 0 1 0
8	1 0 0 0	1 0 1 1	1 1 1 0	1 0 1 1
9	1 0 0 1	1 1 0 0	1 1 1 1	1 1 0 0

- BCD natural: tiene pesos 8421
- BCD Aiken tiene pesos 2421. La razón de esta codificación es la de conseguir simetría entre ciertos números. Ver la simetría en el código Aiken correspondiente a los decimales: 4 y 5, 3 y 6, 2 y 7, 1 y 8, 0 y 9.
- Ejemplo

9 8 3 2 5 10 = 1001 1000 0011 0010 0101BCD-natural

9 8 3 2 5 10 = 1111 1110 0011 0010 1011BCD-Aiken

Código de Representación Decimal BCD

Código BCD AIKEN o 2421

Aquellos números que poseen doble representación, deberá optarse por la que permita invertir un solo dígito respecto a su anterior. Por Ejemplo para 5 base 10 codificar como 1001 en 2421

3	0011
4	<u>0100</u>
5	1011
6	1100
7	1101
8	1110
9	1111

"Código autocomplementario"

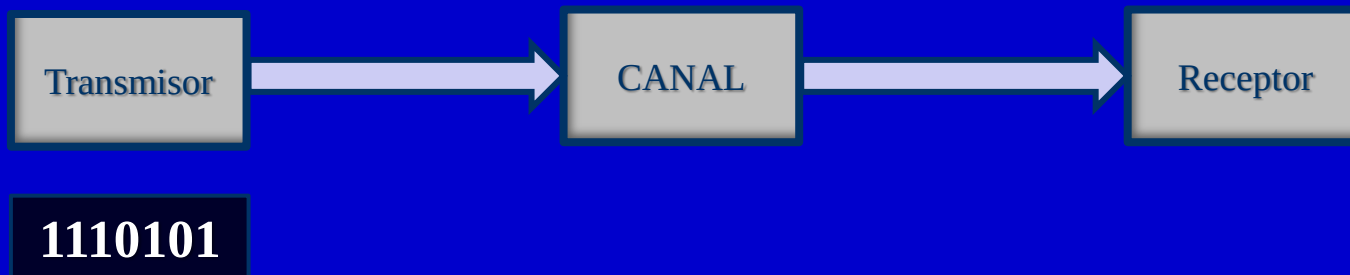
Este código adjudica a cada dígito "1" binario el peso que resulta de la combinación 2421, en lugar del BCD puro que impone un peso igual a 8421. Por ejemplo, el número 7 en BCD 2421 es "1101" y surge de la suma de los dígitos "1" multiplicados por sus pesos, o sea, $2 + 4 + 1 = 7$.

Códigos Redundantes

Un “**Proceso de Comunicación**” puede tener lugar en diversas formas e involucra el movimiento de un flujo de información a través de un CANAL definido en el sistema.

Un “**Sistema de Comunicación Ideal**” se puede representar por tres partes esenciales:

- Transmisor, remitente o fuente
- Canal o medio de almacenamiento
- Receptor

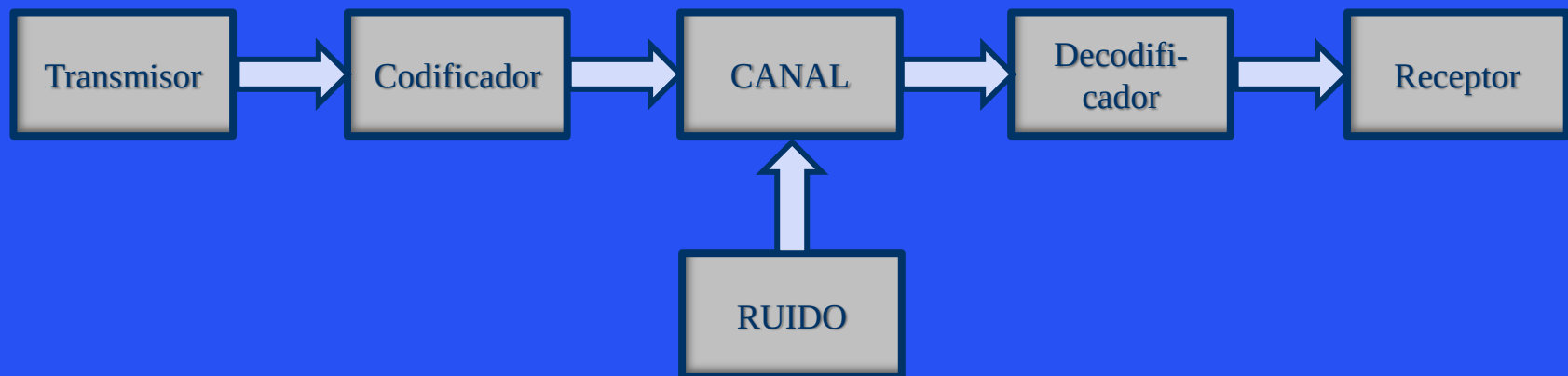


Códigos Redundantes

En un “Sistema de Comunicación Real” un canal de comunicación está sujeto a una a diversidad de perturbaciones que resultan en una distorsión del mensaje que se está transmitiendo. Cualquier alteración de estas se llama **Ruido**.

La forma en la cual el ruido puede aparecer depende de las características particulares de cada canal (Coaxil, Fibra Óptica, Radial, Etc.)

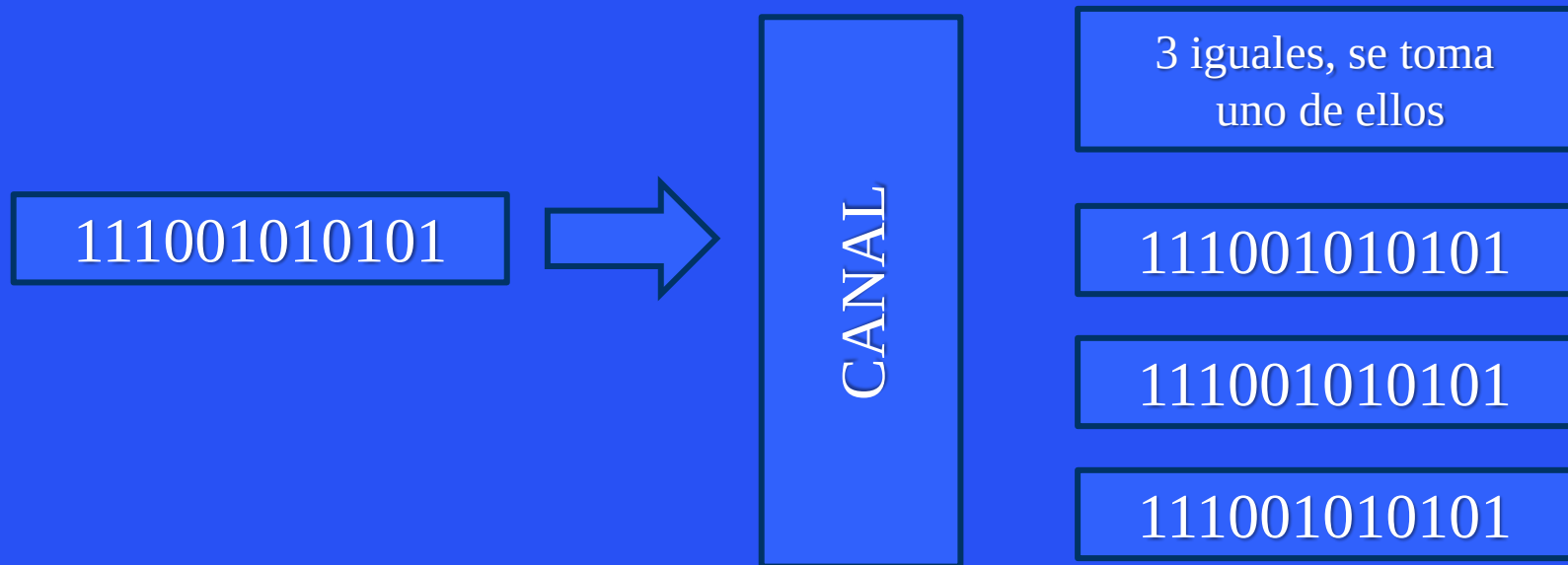
Para mejorar la eficiencia se usan Codificadores y Decodificadores en ambos extremos, que permitan transformar el mensaje para que el receptor pueda detectar y corregir errores



Códigos Redundantes

Códigos de detección y/o corrección de errores

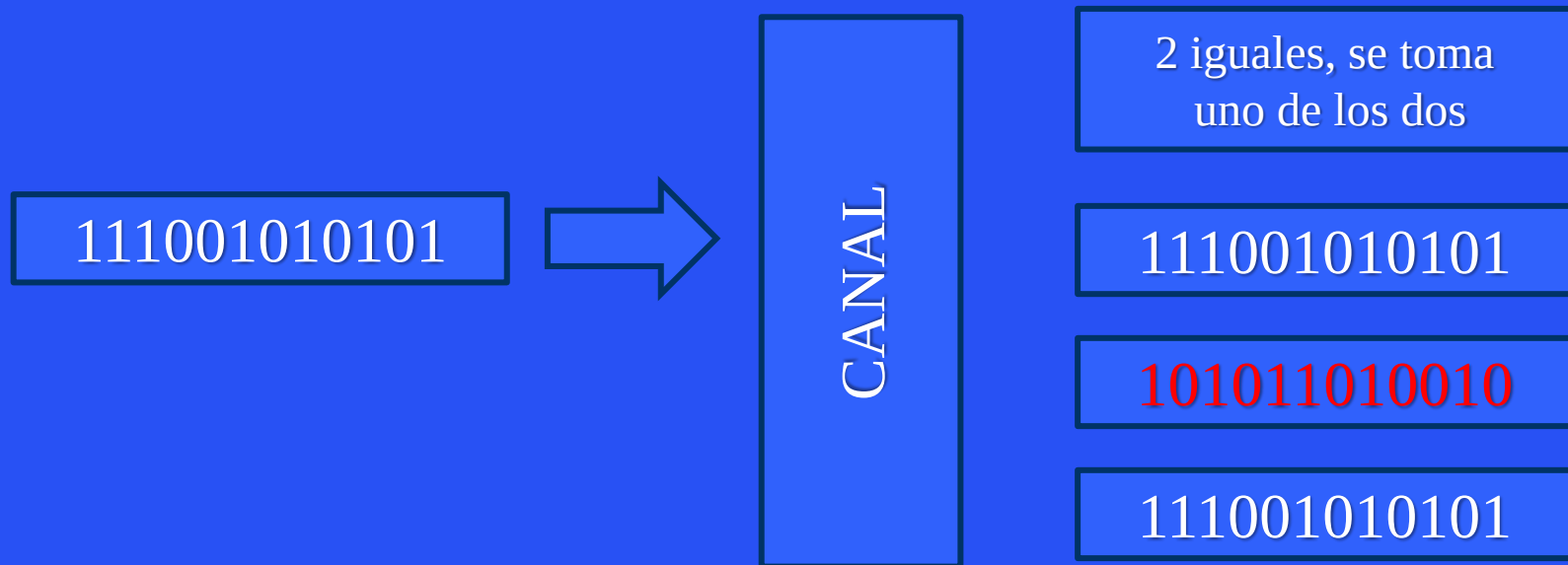
Control Dos entre Tres: Cada mensaje se envía tres veces. El receptor tomará por bueno cuando los tres o dos de tres sean iguales, pidiendo la retransmisión cuando los tres sean distintos



Códigos Redundantes

Códigos de detección y/o corrección de errores

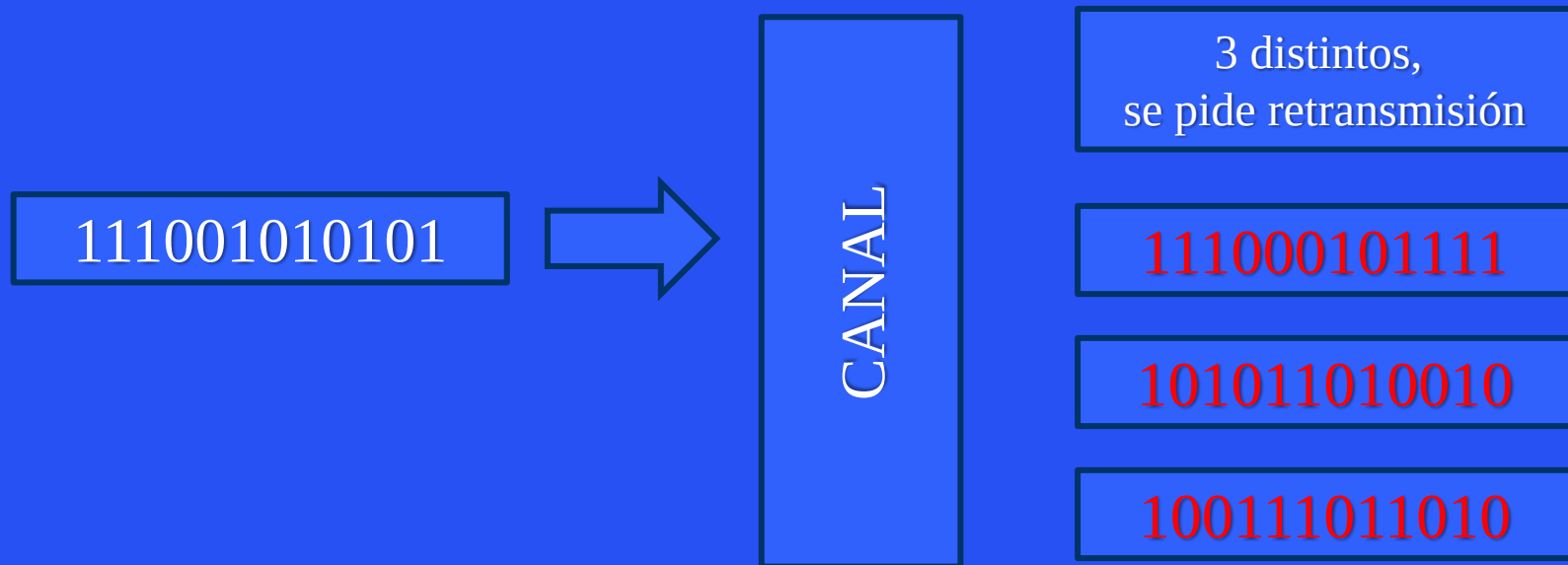
Control Dos entre Tres: Cada mensaje se envía tres veces. El receptor tomará por bueno cuando los tres o dos de tres sean iguales, pidiendo la retransmisión cuando los tres sean distintos



Códigos Redundantes

Códigos de detección y/o corrección de errores

Control Dos entre Tres: Cada mensaje se envía tres veces. El receptor tomará por bueno cuando los tres o dos de tres sean iguales, pidiendo la retransmisión cuando los tres sean distintos



Códigos Redundantes

Códigos de detección y/o corrección de errores

Paridad vertical simple o a nivel carácter

A cada carácter se le agrega un bit de paridad P que cumpla con alguna de las siguientes reglas:

Paridad PAR
De Unos

	1	1	0	1	0	0	0
--	---	---	---	---	---	---	---

P

INFO

Paridad IMPAR
De Unos

	1	1	0	1	0	0	0
--	---	---	---	---	---	---	---

P

INFO

Códigos Redundantes

Códigos de detección y/o corrección de errores

Paridad vertical simple o a nivel carácter

A cada carácter se le agrega un bit de paridad P que cumpla con alguna de las siguientes reglas:

Paridad PAR
De Unos

1	1	1	0	1	0	0	0
---	---	---	---	---	---	---	---

P

INFO

Paridad IMPAR
De Unos

0	1	1	0	1	0	0	0
---	---	---	---	---	---	---	---

P

INFO

Códigos Redundantes

- Los caracteres de los datos se agrupan en bloques de 8 bits cada uno, formando un número de columnas
- Se calcula la paridad de cada fila de bits. El bit de paridad se coloca al final, según
- Se calcula la paridad de cada columna de bits. El bit de paridad se coloca al final,
- El bloque de bits se transmite. El receptor recibe los bits de los datos y el bit de paridad de cada fila y columna.
- La nueva paridad se calcula para cada fila y columna. Si el bit de paridad es diferente al bit de paridad original, se detecta un error.
- La nueva paridad se calcula para cada fila y columna. Si el bit de paridad es diferente al bit de paridad original, se detecta un error.
- Adicionalmente, se calcula la paridad de los bits de los datos y el bit de paridad de cada fila y columna.
- Los caracteres de los datos se agrupan en bloques de 8 bits cada uno, formando un número de columnas
- Se calcula la paridad de cada fila de bits. El bit de paridad se coloca al final, según
- Se calcula la paridad de cada columna de bits. El bit de paridad se coloca al final,
- El bloque de bits se transmite. El receptor recibe los bits de los datos y el bit de paridad de cada fila y columna.
- La nueva paridad se calcula para cada fila y columna. Si el bit de paridad es diferente al bit de paridad original, se detecta un error.
- La nueva paridad se calcula para cada fila y columna. Si el bit de paridad es diferente al bit de paridad original, se detecta un error.
- Adicionalmente, se calcula la paridad de los bits de los datos y el bit de paridad de cada fila y columna.
- Los caracteres de los datos se agrupan en bloques de 8 bits cada uno, formando un número de columnas
- Se calcula la paridad de cada fila de bits. El bit de paridad se coloca al final, según
- Se calcula la paridad de cada columna de bits. El bit de paridad se coloca al final,
- El bloque de bits se transmite. El receptor recibe los bits de los datos y el bit de paridad de cada fila y columna.
- La nueva paridad se calcula para cada fila y columna. Si el bit de paridad es diferente al bit de paridad original, se detecta un error.
- La nueva paridad se calcula para cada fila y columna. Si el bit de paridad es diferente al bit de paridad original, se detecta un error.
- Adicionalmente, se calcula la paridad de los bits de los datos y el bit de paridad de cada fila y columna.

Paridad Horizontal	Bit1	Bit2	Bit3	Bit4	Bit5	Bit6	Bit7	Bit8	0
0	0	1	1	0	0	1	0	1	
1	1	0	1	1	1	0	0	1	
0	1	0	0	1	0	1	0	1	
1	0	0	1	1	1	0	0	0	
1	1	0	0	1	0	1	1	1	
1	0	0	1	0	1	0	1	0	
0	1	1	0	0	1	1	0	0	Paridad Vertical

A continuación se muestra un ejemplo en el que se chequea la paridad de un bloque de 48 bits, distribuido en 6 filas de 8 bits cada una. Se usa paridad par.

Códigos Redundantes

Paridad horizontal y vertical a nivel de bloque o CÓDIGO ENTRELAZADO

EJEMPLO

de UN

- La
- Si
- Se

co

pa
int

	I_0	I_1	I_2	I_3	PH
	0	1	0	1	0
	0	1	1	1	1
	1	1	0	1	1
	1	0	1	1	1
	1	1	1	0	1
	1	0	1	1	1
	0	1	1	0	0
PV	0	1	1	1	1

	I_0	I_1	I_2	I_3	PH
	0	1	0	1	0
	0	0	1	1	1
	1	1	0	1	1
	1	0	1	1	1
	1	1	1	0	1
	1	0	1	1	1
	0	1	1	0	0
PV	0	1	1	1	1

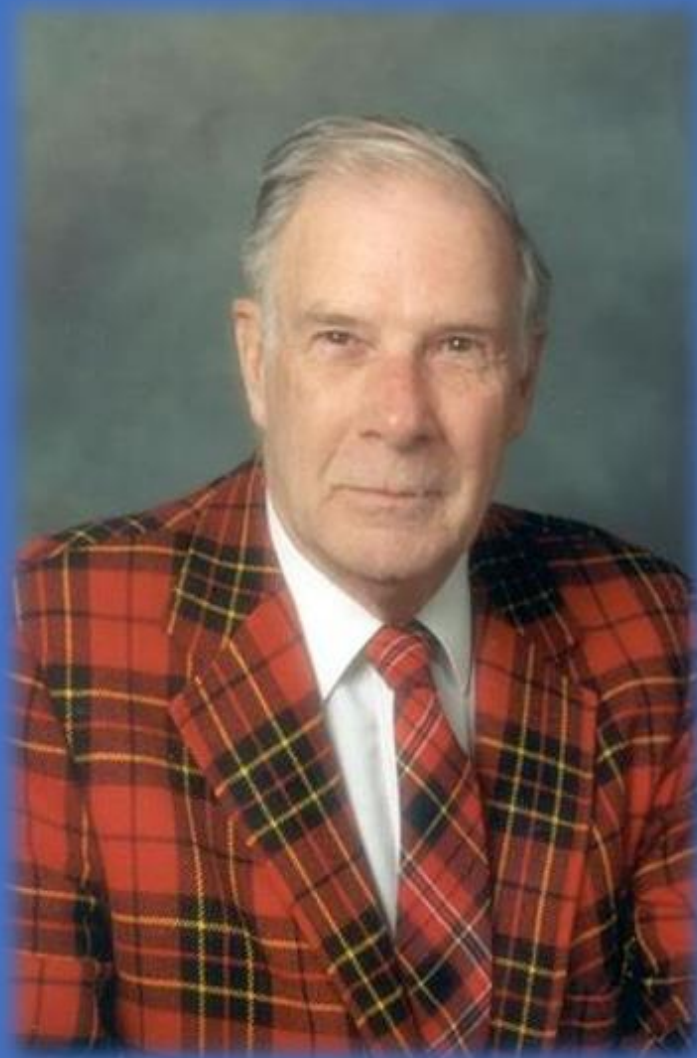
PAR

0

NO

i
la

Códigos Redundantes



RICHARD WESLEY HAMMING

Matemático estadounidense de gran importancia en el área de la informática y de las telecomunicaciones, nació en Chicago, Illinois, el 11 de febrero de 1915. Estudió su licenciatura en la Universidad de Chicago en 1937, en 1939 realiza un master en la Universidad de Nebraska y se doctoró en la Universidad Urbana-Champaign de Illinois en 1942.

Mientras se desarrollaba la Segunda Guerra Mundial, fue profesor en la Universidad de Louisville, trabajo que abandonaría para integrarse en 1945 en el proyecto Manhattan. Allí desarrolló su trabajo programando una de las primeras calculadoras numéricas electrónicas, para determinar la solución a algunas ecuaciones proporcionadas por los físicos del proyecto. El objetivo del programa era descubrir si la detonación de una bomba atómica podría incendiar la atmósfera. Entre los años 1946-1976, trabajó en los laboratorios Bell, en donde colaboró con Claude E. Shannon. El 23 de julio de 1976 se trasladó a la Naval Postgraduate School, en donde trabajó como profesor adjunto hasta 1997, llegando a ser Professor Emeritus. Fue fundador y presidente de la Association for Computing Machinery. Murió en Monterey, California el 7 de enero de 1998.

Código de Hamming

Desde que Claude Shanon desarrollo su teoría de la información, la transmisión de la información digital presenta un reto constante para los ingenieros de comunicaciones.

Es sabido que los medios de transmisión (Canales) pueden corromper y dañar los datos, por lo que es necesario desarrollar un método que permitan detectar y corregir errores.

Uno de ellos es el desarrollado por Richard Wesley Hamming, que permite detectar y corregir un solo error de una palabra de información codificada por este método.

El código Hamming permite detección y corrección de los datos enviados por un canal susceptible a ruido, este método se utiliza en canales donde la retransmisión de un mensaje puede congestionar el canal, este método se utiliza comúnmente en redes de Wi-Fi para la transmisión de mensajes, su estudio nos mostrará como detecta y corrige errores de un bit, aunque puede ser escalado para que detecte más errores mejorando la eficiencia en el canal.

Códigos Redundantes

Código de Hamming

Es un código que se utiliza en la detección y corrección de errores que se producen en la transmisión de códigos binarios, la palabra de código se conforma por los bits de comprobación y los bits de información. Este

Conceptos Básicos

- Código binario: Es una representación unívoca de las cantidades, de tal forma que a cada una de éstas se le asigna una combinación de símbolos binarios.
- Distancia entre dos combinaciones binarias: Viene dada por el número de bits que hay que cambiar en una de ellas para obtener la otra.
- Distancia mínima de un código: Es la menor de las distancias entre dos combinaciones binarias cualesquiera pertenecientes a dicho código.

Distancia de Hamming

El código Hamming se genera a partir de una distancia mínima de 3, ya que una distancia de 1 o 2 es inconveniente para poder generar un código para detectar un error.

La distancia mínima de Hamming está dada por la siguiente ecuación:

$$D_m = 2X + 1$$

Donde D_m es la distancia mínima de un código para permitir la corrección de datos y X las líneas de datos.

Si se tuviese una distancia de 1 no tendríamos valores para saber si hay o no un error.

Si se tienen una distancia de 2 se podría lograr un sistema de corrección de error con un bit de paridad pero el problema que presenta es que un sistema con distancia 2 puede tener datos erróneos que pueden pasar como datos correctos.

Con la distancia de 3 se tiene que los códigos no van a tener datos o resultados similares por ende se puede detectar el error y se puede desarrollar un sistema para su corrección como el Hamming

Variables para la generación de Hamming

Se deben tener en cuenta tres variables:

n: número de bits de información a transmitir.

p: número de bits de paridad par generados en el transmisor, o sea, número de bits que añadimos al código inicial.

c: número de bits detectores de paridad par generados por el receptor.

El valor de $p = c$, ya que los bits de paridad generados tanto en el transmisor como en el receptor debe ser la misma.

Códigos Redundantes

Distancia de Hamming

$$D_m = 2X + 1$$

Donde D_m es la distancia mínima de un código para permitir la corrección de datos y X las líneas de datos.

Si se tuviese una distancia de 1 no tendríamos valores para saber si hay o no un error.

Si se tienen una distancia de 2 se podría lograr un sistema de corrección de error con un bit de paridad pero el problema que presenta es que un sistema con distancia 2 puede tener datos erróneos que pueden pasar como datos correctos.

Con la distancia de 3 se tiene que los códigos no van a tener datos o resultados similares por ende se puede detectar el error y se puede desarrollar un sistema para su corrección como el Hamming

Códigos Redundantes

Distancia de Hamming

$$D_m = 2X + 1$$

$$D_m = 2 * 0 + 1 = 1$$

Distancia HAMMING = 1

Mensaje	
0	Válido
1	Válido

Se debe modificar 1 bit

$$D_m = 2 * 1 + 1 = 3$$

Distancia HAMMING = 2

Mensaje		
0	0	Válido
0	1	Invalido
1	0	Invalido
1	1	Válido

Se pueden detectar errores de 1 bit

(Caso bit Paridad)

$$D_m = 2 * 3 + 1 = 7$$

Distancia HAMMING = 3

Mensaje			
0	0	0	Válido
0	0	1	Invalido
0	1	0	Invalido
0	1	1	Invalido
1	0	0	Invalido
1	0	1	Invalido
1	1	0	Invalido
1	1	1	Válido

Detecta y corrige errores de 1 bit

Códigos Redundantes

Distancia de Hamming entre dos Códigos

Se denomina Distancia de Hamming entre dos códigos de igual longitud, al número de símbolos que se deben alterar para pasar de una configuración válida a otra. Es decir, el número de bits en que difieren dos configuraciones binarias válidas de igual longitud.

Se puede calcular la distancia sumando ambas cadenas pero sin considerar el acarreo y finalmente contando la cantidad de unos obtenidos

Por Ejemplo:

$$\begin{array}{l} A=1101101011 \\ B=\underline{1010110011} \end{array} + \\ \underline{011011000} \rightarrow \text{Distancia} = 5$$

$$\begin{array}{l} A= 1011010111 \\ B= \underline{1101100001} \end{array} + \\ \underline{0110110110} \rightarrow \text{Distancia} = 6$$

Códigos Redundantes

Distancia de Hamming

Distancia	Longitud Total (n)	Longitud Información (k)	Notación (n,k)	Eficiencia sobre Canal	Detección	Corrección int((m-1)/2)
m	$(2^m)-1$	$(2^m)-1-m$	(n,k)	k/n	(m-1)	(m-1)/2
1	1	0	(1,0)	0,0%	0	0
2	3	1	(3,1)	33,3%	1	0
3	7	4	(7,4)	57,1%	2	1
4	15	11	(15,11)	73,3%	3	1
5	31	26	(31,26)	83,9%	4	2
6	63	57	(63,57)	90,5%	5	2
7	127	120	(127,120)	94,5%	6	3
8	255	247	(255,247)	96,9%	7	3
9	511	502	(511,502)	98,2%	8	4
10	1023	1013	(1023,1013)	99,0%	9	4
11	2047	2036	(2047,2036)	99,5%	10	5
12	4095	4083	(4095,4083)	99,7%	11	5

Con distancia 1 no se detecta ni se corrigen errores

Con distancia 2 se detecta un error sin posibilidad de corrección (paridad simple)

La eficiencia de este código comienza a partir de distancias igual a 3 o superior

Códigos Redundantes

Código de Hamming

Este código permite detectar y corregir un solo errores producido en una transmisión con sólo agregar p bits de paridad, de forma que se cumpla la relación siguiente: $2^p \geq i + p + 1$ donde:

- i = Cantidad de bits de Información a Transmitir
- P = Cantidad de bits de paridad

Para satisfacer la relación enunciada antes:

$i = 4$ para que la relación $2^p \geq i + p + 1$ se cumpla debe ser
 $p = ?$ $p = 3$, luego $2^3 \geq 4 + 3 + 1$.

La tabla siguiente es la guía para determinar la distribución y el cálculo de los bits de paridad y su chequeo luego de la transmisión de hasta 11 dígitos binarios.

Código de Hamming

Posición mensaje	7	6	5	4	3	2	1
	i_3	i_2	i_1	p_2	i_0	p_1	p_0

La ubicación de los bits de paridad en el mensaje es arbitrario y responde a las posiciones Potencias de 2

	p_2	p_1	p_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Cada bit de paridad controla la posición que coincide con un 1 en su columna en la tabla binaria. Es decir:

p_0 controla las posiciones: 1, 3, 5 y 7

p_1 controla las posiciones: 2, 3, 6 y 7

p_2 controla las posiciones: 4, 5, 6 y 7

Código de Hamming

Posición mensaje	7	6	5	4	3	2	1
	i_3	i_2	i_1	p_2	i_0	p_1	p_0

La ubicación de los bits de paridad en el mensaje es arbitrario y responde a las posiciones Potencias de 2

	p_2	p_1	p_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Cada bit de paridad controla la posición que coincide con un 1 en su columna en la tabla

binaria. Es decir:

p_0 controla las posiciones: 1, 3, 5 y 7

p_1 controla las posiciones: 2, 3, 6 y 7

p_2 controla las posiciones: 4, 5, 6 y 7

Código de Hamming

Posición mensaje	7	6	5	4	3	2	1
	i_3	i_2	i_1	p_2	i_0	p_1	p_0

La ubicación de los bits de paridad en el mensaje es arbitrario y responde a las posiciones Potencias de 2

	p_2	p_1	p_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Cada bit de paridad controla la posición que coincide con un 1 en su columna en la tabla

binaria. Es decir:

p_0 controla las posiciones: 1, 3, 5 y 7

p_1 controla las posiciones: 2, 3, 6 y 7

p_2 controla las posiciones: 4, 5, 6 y 7

Código de Hamming

Posición mensaje	7	6	5	4	3	2	1
	i_3	i_2	i_1	p_2	i_0	p_1	p_0

La ubicación de los bits de paridad en el mensaje es arbitrario y responde a las posiciones Potencias de 2

	p_2	p_1	p_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Cada bit de paridad controla la posición que coincide con un 1 en su columna en la tabla

binaria. Es decir:

p_0 controla las posiciones: 1, 3, 5 y 7

p_1 controla las posiciones: 2, 3, 6 y 7

p_2 controla las posiciones: 4, 5, 6 y 7

Código de Hamming

Control y corrección

Cuando el mensaje es enviado, el receptor calcula el valor de los bits de control de la siguiente forma:

$$c_0 = 1 \oplus 3 \oplus 5 \oplus 7$$

$$c_1 = 2 \oplus 3 \oplus 6 \oplus 7$$

$$c_2 = 4 \oplus 5 \oplus 6 \oplus 7$$

Si el bit de control es igual a 0 es correcto, si es 1 es incorrecto. La posición del error se determina según los valores obtenidos en los bits de control ordenados en forma decreciente, es decir c_2 , c_1 y c_0 . Si todos los bits de control son igual a cero no se detecta error en el mensaje.

Código de Hamming

Control y corrección

Cuando el mensaje es enviado, el receptor calcula el valor de los bits de control de la siguiente forma:

EJEMPLO

$$c_0 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

$$c_1 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$c_2 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$$

Si el bit de control es igual a 0 es correcto, si es 1 es incorrecto. La posición del error se determina según los valores obtenidos en los bits de control ordenados en forma decreciente, es decir c_2 , c_1 y c_0 . Si todos los bits de control son igual a cero no se detecta error en el mensaje.

Código de Hamming

Control y corrección

Cuando el mensaje es enviado, el receptor calcula el valor de los bits de control de la siguiente forma:

EJEMPLO

$c_0 = 1 \oplus 0 \oplus 1 \oplus 1 =$	1
$c_1 = 1 \oplus 1 \oplus 0 \oplus 0 =$	0
$c_2 = 1 \oplus 1 \oplus 1 \oplus 1 =$	0

↑

Posición Error:
001

Si el bit de control es igual a 0 es correcto, si es 1 es incorrecto. La posición del error se determina según los valores obtenidos en los bits de control ordenados en forma decreciente, es decir c_2 , c_1 y c_0 . Si todos los bits de control son igual a cero no se detecta error en el mensaje.

Código de Hamming

Ejercicio Propuesto

Se desea Transmitir en ASCII la letra B desde la CPU al Hd de la PC.
Codificar en Hamming Dicha Información

Si $i = 7$ y $2^p \geq i + p + 1$ entonces $p = 4$ ya que $2^4 \geq 7 + 4 + 1$.

B (ASCII)							1	0	0	0	0	1	0
							7	6	5	4	3	2	1
11	10	9	8	7	6	5	4	3	2	1			
i_6	i_5	i_4	p_3	i_3	i_2	i_1	p_2	i_0	p_1	p_0			

Código de Hamming

Ejercicio Propuesto

Se desea Transmitir en ASCII la letra B desde la CPU al Hd de la PC.
Codificar en Hamming Dicha Información

Si $i = 7$ y $2^p \geq i + p + 1$ entonces $p = 4$ ya que $2^4 \geq 7 + 4 + 1$.

1	0	0		0	0	1		0		0
11	10	9	8	7	6	5	4	3	2	1
i_6	i_5	i_4	p_3	i_3	i_2	i_1	p_2	i_0	p_1	p_0

p_0 controla : 1, 3, 5, 7, 9, 11

Código de Hamming

Ejercicio Propuesto

Se desea Transmitir en ASCII la letra B desde la CPU al Hd de la PC.
Codificar en Hamming Dicha Información

Si $i = 7$ y $2^p \geq i + p + 1$ entonces $p = 4$ ya que $2^4 \geq 7 + 4 + 1$.

1	0	0		0	0	1		0	1	0
11	10	9	8	7	6	5	4	3	2	1
i_6	i_5	i_4	p_3	i_3	i_2	i_1	p_2	i_0	p_1	p_0

p0 controla : 1, 3, 5, 7, 9, 11
p1 controla : 2, 3, 6, 7, 10, 11

Código de Hamming

Ejercicio Propuesto

Se desea Transmitir en ASCII la letra B desde la CPU al Hd de la PC.
Codificar en Hamming Dicha Información

Si $i = 7$ y $2^p \geq i + p + 1$ entonces $p = 4$ ya que $2^4 \geq 7 + 4 + 1$.

1	0	0		0	0	1	1	0	1	0
11	10	9	8	7	6	5	4	3	2	1
i_6	i_5	i_4	p_3	i_3	i_2	i_1	p_2	i_0	p_1	p_0

p0 controla : 1, 3, 5, 7, 9, 11

p1 controla : 2, 3, 6, 7, 10, 11

p2 controla : 4, 5, 6, 7

Código de Hamming

Ejercicio Propuesto

Se desea Transmitir en ASCII la letra B desde la CPU al Hd de la PC.
Codificar en Hamming Dicha Información

Si $i = 7$ y $2^p \geq i + p + 1$ entonces $p = 4$ ya que $2^4 \geq 7 + 4 + 1$.

1	0	0	1	0	0	1	1	0	1	0
11	10	9	8	7	6	5	4	3	2	1
i_6	i_5	i_4	p_3	i_3	i_2	i_1	p_2	i_0	p_1	p_0

p0 controla : 1, 3, 5, 7, 9, 11

p1 controla : 2, 3, 6, 7, 10, 11

p2 controla : 4, 5, 6, 7

P3 controla : 8, 9, 10, 11

Código de Hamming

Ejercicio Propuesto

Se desea Transmitir en ASCII la letra B desde la CPU al Hd de la PC.
Codificar en Hamming Dicha Información

Si $i = 7$ y $2^p \geq i + p + 1$ entonces $p = 4$ ya que $2^4 \geq 7 + 4 + 1$.

1	0	0	1	0	0	1	1	0	1	0
<i>i</i> ₆	<i>i</i> ₅	<i>i</i> ₄	<i>p</i> ₃	<i>i</i> ₃	<i>i</i> ₂	<i>i</i> ₁	<i>p</i> ₂	<i>i</i> ₀	<i>p</i> ₁	<i>p</i> ₀

Mensaje Codificado en Hamming

Código de Hamming

Ejercicio Propuesto

Del lado del receptor se recibe el mensaje codificado con un error en el bit de posición siete (7)

1	0	0	1	1	0	1	1	0	1	0
<i>11</i>	<i>10</i>	<i>9</i>	<i>8</i>	<i>7</i>	<i>6</i>	<i>5</i>	<i>4</i>	<i>3</i>	<i>2</i>	<i>1</i>
<i>i₆</i>	<i>i₅</i>	<i>i₄</i>	<i>p₃</i>	<i>i₃</i>	<i>i₂</i>	<i>i₁</i>	<i>p₂</i>	<i>i₀</i>	<i>p₁</i>	<i>p₀</i>

c0 ORX Posiciones: 1, 3, 5, 7, 9, 11

c1 ORX Posiciones: 2, 3, 6, 7, 10, 11

c2 ORX Posiciones: 4, 5, 6, 7

c3 ORX Posiciones: 8, 9, 10, 11

1
1
1
0



Posición Error:
0111 (7)

Código de Hamming

Ejercicio Propuesto

Del lado del receptor se recibe el mensaje codificado con un error en el bit de posición siete (7)

Mensaje Corregido del lado del Receptor

1	0	0	1	1	0	1	1	0	1	0
11	10	9	8	7	6	5	4	3	2	1
i_6	i_5	i_4	p_3	i_3	i_2	i_1	p_2	i_0	p_1	p_0

c0 ORX Posiciones: 1, 3, 5, 7, 9, 11

c1 ORX Posiciones: 2, 3, 6, 7, 10, 11

c2 ORX Posiciones: 4, 5, 6, 7

c3 ORX Posiciones: 8, 9, 10, 11

1
1
1
0



Posición Error:
0111 (7)

Fin